

Practical ASR & TTS Lab

with Open-Source Burmese Dataset SLR80

Wav2Vec2-CTC for ASR ■ TransformerTTS + Guided Attention for TTS

Dr. Ye Kyaw Thu, Language Understanding Lab., Myanmar

Dataset: <https://www.openslr.org/80/>

SLR80 — Crowdsourced Burmese (Myanmar) Speech Corpus

June 2026

Outline

- 1 Dataset
- 2 ASR Code
- 3 ASR Experiments and Results
- 4 TTS Code
- 5 TTS Experiments and Results
- 6 Summary
- 7 References

SLR80 — Crowdsourced Burmese Speech Corpus

Overview

- **Source:** OpenSLR 80
<https://www.openslr.org/80/>
- **Language:** Burmese (Myanmar)
- **Content:** Female single-speaker read speech
- **License:** CC BY-SA 4.0
- **Size:** ~ 904 MB (zip), ~ 2.5 h

Files

- `line_index_female.tsv` — ID ↔ transcript
- `bur_XXXXXXXXXXXXXXXXX.wav` — 2 530 WAV files

Sample TSV entries

bur_7865_1250917969 ပြီးတော့ တရုတ် နဲ့လည်း
ချစ်ကြည်ရင်းနှီးတဲ့ ဆက်ဆံရေး ရှိတယ်
bur_8698_6883351313 အဲ့ဒီ ဝေဖန်မှုတွေ နဲ့ ပတ်သက်
လို့ ဘယ်လို တုံ့ပြန်ချင်ပါသလဲ

Challenge

Only ~ 2.5 h — *very low-resource* for both ASR and TTS.

Data Splits & Statistics

Train / Dev / Test Split (80/10/10)

Split	Utterances	%
Train	2 024	80.0
Dev	253	10.0
Test	253	10.0
Total	2 530	100

TTS — Mel Spectrogram Stats

Statistic	Value
Vocab size (chars)	66
Mel frames (mean)	504 ± 150
Frames / char (mean)	7.3 ± 1.6
Mel norm range	$[-1.0, +1.0]$
Audio SR	22 050 Hz

ASR — Vocab

Token type	Count
Special ([PAD], [UNK],)	3
Unique Myanmar chars	61
Total	64

Wav2Vec & Wav2Vec2

- **Wave2Vec** (Schneider et al., 2019): A self-supervised learning framework for speech. Instead of needing transcribed audio (which Myanmar lacks in large quantities), it learns representations by masking parts of the audio and trying to predict them. It uses a "contrastive predictive coding" (CPC) objective.
- **Wave2Vec 2.0** (Baevski et al., 2020): The massive upgrade. It simplified the architecture by removing the contrastive loss and replacing it with a noise-resistant training objective. It masks continuous spans of audio (e.g., 10ms to 30ms) and trains a Transformer encoder to fill in the blanks. This yielded representations that could be fine-tuned for ASR with very little labeled data.

Facebook/Wav2Vec2-base

- **facebook/wav2vec2-base:** This is the specific checkpoint I used.
- The "base" variant consists of a 7-layer CNN feature extractor followed by a 12-layer Transformer encoder with 12 attention heads (totaling ~90M parameters).
- I chose this because it is small enough to fine-tune on a single GPU with only ~2.5 hours of Burmese data, avoiding overfitting compared to the "large" (300M) or "xlarge" (1B) variants.

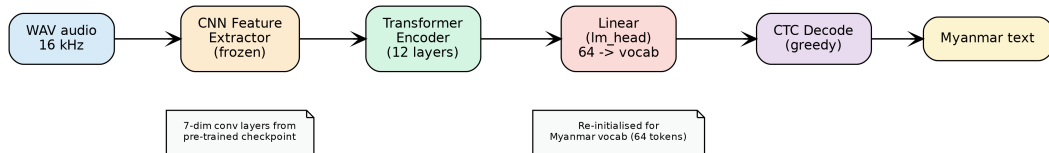
CTC (Connectionist Temporal Classification)

- **What it is:** An algorithm used to train neural networks for sequence-to-sequence tasks (like ASR) without needing frame-by-frame alignments between the audio and the text. It adds a special "blank" token to the vocabulary.
- **How it works:** Audio is processed frame-by-frame. The network outputs a probability distribution over characters (plus blank) for every single frame. During decoding, CTC applies two rules:
 - 1 **Collapse:** Remove consecutive duplicate characters (e.g., "k-k-k" $\rightarrow$ "k").
 - 2 **Remove blanks:** Delete all blank tokens.

Why Wav2Vec2-CTC for Low-Resource Myanmar?

- **Pre-trained representations:** facebook/wav2vec2-base provides strong acoustic features from 960 h of unlabeled audio; only ~ 2.5 h of Burmese needed for fine-tuning.
- **CTC objective:** no alignment required — each audio frame is mapped to a character (or blank) independently.
- **Character-level tokeniser:** Myanmar is an abugida; each Unicode code point is a meaningful unit. Small vocab (~ 64) $\Rightarrow$ easy CTC alignment, no OOV.
- **Frozen feature encoder:** CNN layers kept frozen to prevent over-fitting on only 2024 utterances.

ASR — Architecture Diagram



Key: Custom Tokeniser

Stock English tokeniser **replaced** with `Wav2Vec2CTCTokenizer` built from `myanmar_vocab.json`. Spaces → |.

Key: Version-Adaptive Code

Runtime introspection of `TrainingArguments` / `Trainer` signatures for compatibility across transformers 4.x–5.x.

ASR — Key Code Techniques

1. Custom CTC Tokeniser

```
1 vocab = build_myanmar_vocab(texts)
2 # {"[PAD]":0, "[UNK]":1, "|":2,
3 # "...":3, ...} # 64 tokens
4
5 tokenizer = Wav2Vec2CTCTokenizer(
6     vocab_path,
7     unk_token="[UNK]",
8     pad_token="[PAD]",
9     word_delimiter_token="|",
10 )
11 processor = Wav2Vec2Processor(
12     feature_extractor, tokenizer)
```

2. CTC Greedy Decode

```
1 def _ctc_decode(ids):
2     out, prev = [], None
3     for tid in ids:
4         if tid != prev: # collapse
5             if tid != pad_id: # rm blank
6                 out.append(tid)
7             prev = tid
8     # map id -> char, "/" -> " "
9     return "".join(chars).strip()
```

3. Myanmar CER

Levenshtein DP on *character* sequences. CER
= $(S + D + I) / N_{\text{ref chars}}$
Primary metric (word boundaries inconsistent).

ASR — Step 1: Download & Setup

Step 1: Download Dataset

```
1 time ./download_dataset.sh
2 # wget https://www.openslr.org/resources/80/my_mm_female.zip
3 # unzip my_mm_female.zip -> 2530 .wav files
4 # Real time: 1m 20s
```

Step 2-3: Install Dependencies & Verify

```
1 pip install -r requirements.txt
2 python myanmar_asr.py --check_deps
3 # TensorFlow 2.19.0 available. All dependencies satisfied.
```

Download Output

```
my_mm_female.zip 100%==> 904.26M 13.5MB/s in 69s
```

ASR — Step 4: Data Preparation

Command

```
1 time python myanmar_asr.py --stage prep \  
2   --data_dir ./data/slr80 \  
3   --output_dir ./asr_output \  
4   --train_ratio 0.80 --dev_ratio 0.10
```

Output Log

```
Audio found flat in: data/slr80/ (2530 wav files)  
TSV parse: 2530 valid (0 missing, 0 malformed)  
  train: 2024 utterances → train.json  
  dev:   253 utterances → dev.json  
  test:  253 utterances → test.json  
Myanmar vocab: 64 tokens → myanmar_vocab.json  
Real time: 1.01s
```

Generated Artifacts

- train.json / dev.json / test.json
- myanmar_vocab.json — char → id mapping
- Vocab from **train only** (no test leakage)

ASR — Step 5: Training (Wav2Vec2-CTC Fine-Tuning)

Command

```
1 time python myanmar_asr.py --stage train \  
2   --output_dir ./asr_output \  
3   --model_name facebook/wav2vec2-base \  
4   --epochs 20 --lr 1e-4 \  
5   --batch_size 8 --grad_accum 2 \  
6   --fp16 --early_stop_patience 5
```

Hyperparameters

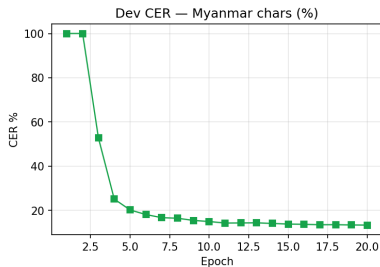
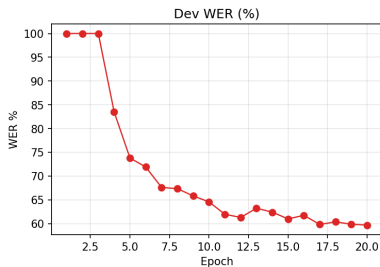
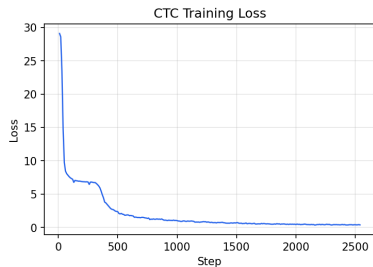
Base model	facebook/wav2vec2-base
Vocab size	64 (Myanmar chars)
Epochs	20
LR	1×10^{-4}
Batch / device	8
Grad accumulation	2 (eff. batch=16)
Warmup ratio	0.10
FP16	✓
Feature encoder	Frozen
Early stopping	patience=5

Training Summary

- `ignore_mismatched_sizes=True`: only `lm_head` re-initialised (64-way)
- All 12 Transformer encoder layers from pre-trained checkpoint
- Version-adaptive: works with transformers 4.x and 5.x
- Best model by **lowest CER** on dev

ASR — Learning Curves

ASR Learning Curves — Myanmar SLR80



Left: CTC training loss decreases steadily. **Centre:** Dev WER converges. **Right:** Dev CER (Myanmar chars) — primary metric.

ASR — Step 6: Evaluation Results

Test Set Performance

Metric	Value
CER (Myanmar)	13.29%
WER	58.19%
Total chars	17 808
Total words	2 270
Substitutions	1 172
Deletions	617
Insertions	578
Failed decodes	0

CER is the primary metric for Myanmar because word boundaries are written inconsistently.

Evaluation Command

```
1 time python myanmar_asr.py --stage eval \  
2   --output_dir ./asr_output  
3 # Real time: 7.8s
```

Error Analysis

Dominant error: Substitutions (1 172) > Deletions (617) > Insertions (578).

Confusable Myanmar characters (e.g. **ခ/ဂ**, **ည/ဉ**) cause most substitution errors.

ASR — Example Decoding Results

Top-3 Worst Utterances by CER

ID	Reference	Hypothesis	CER
bur_7712	စန္ဒာကေတို့ အိုမီဂါတို့ လိုက် လုပ်ကြသည်	ရော စိတာကေးတို့ ဝန်းမီ ကတို့ လိုက် လုပ်ကြသည်	35.9%
bur_0366	ထို အဆင့်များ ခွင့်ပြု လျှင် ပုံနှိပ်စက် တင်၍ ပုံနှိပ်ရသည်	ဒဂိုစစ် များခွင့်ပြု၍ ရှောင် ပုံနှိပ်စက်တင်တွေ ပုံနှိပ်ရသည်	33.9%
bur_9135	ကရင် ပြည်နယ် မှာတော့ နေရောင် မမြင်ရဘဲ	ကရင် ပြည်နယ် မာတော့ နေယောက် မမီးယပဲ	34.0%

Observation

Worst cases involve **rare consonant clusters** and **Pali/Sanskrit loanwords** (စန္ဒာ, ပုံနှိပ်) under-represented in the small training set (~ 2 h).

TTS — Modeling Approach & Motivation

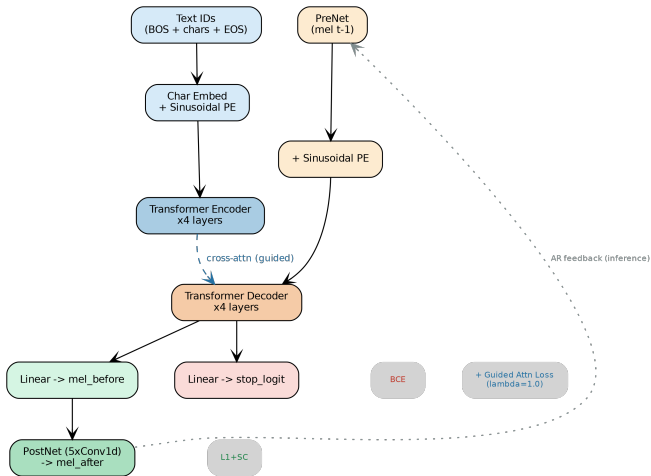
Why Autoregressive TransformerTTS?

- **v1–v3 (non-AR + Gaussian upsampling) failed:** each mel frame receives a *soft mixture* of character embeddings $\Rightarrow$ model predicts conditional mean = flat spectrum = wind noise . Dev loss **plateaued at 1.06**.
- **v4 (AR decoder) fixed wind but collapsed at inference:** exposure bias + unconstrained cross-attention $\Rightarrow$ stop token fired after ~ 11 frames regardless of input text length .
- **v5 (current):** adds **Guided Attention Loss** [5] + length-aware inference floor $\Rightarrow$ forces monotonic left-to-right alignment.

Key Insight

With only ~ 2000 utterances, vanilla cross-attention **never discovers** monotonic alignment on its own and finds a shortcut: ignore text $\rightarrow$ predict stop immediately. Guided attention provides an **explicit gradient** toward the diagonal.

TTS — Architecture Diagram (v5)



Total parameters: **45.4 M**. PreNet dropout kept **active at inference** (Tacotron2 finding).

TransformerTTS

- **The Flow:** TransformerTTS follows the standard Tacotron paradigm but replaces LSTMs with Transformer blocks.
 - ① **Text Encoder:** Converts character IDs into embeddings, adds positional info, and passes them through 4 Transformer encoder layers to create a rich textual representation.
 - ② **Cross-Attention:** The decoder "looks" at the encoder output to figure out what to say.
 - ③ **Autoregressive Decoder:** Generates the mel spectrogram one frame at a time.
- **Why this architecture?** It allows the model to learn complex, long-range dependencies in the text (e.g., Myanmar vowel stacking, tone marks) better than older RNN-based models.

Why & What Mel Spec

- **Why Mel instead of raw audio?** Raw audio has millions of samples per second and massive variance. The Mel spectrogram compresses this into a 2D image-like matrix (Time $\times$ Frequency) that captures exactly what the human ear cares about: pitch and timbre.
- **What it is:** A visual representation of the frequencies of a sound over time. It applies the Mel scale—a perceptual scale of pitches judged by listeners to be equal in distance from one another.

Why & What Mel Spec

- **In my code:** I extract 80 Mel frequency bins using an FFT size of 1024 and a hop length of 256 (each frame represents ~ 11.6 ms of audio). The power spectrum is converted to decibels (log scale), yielding values between -80 dB (silence) and 0 dB (loudest). And then linearly normalize this to $[-1.0, +1.0]$ so the neural network can process it easily.

Spectrogram — Standard vs. Mel

Feature	Standard Spectrogram	Mel Spectrogram
Frequency Scale	Linear (Hz)	Non-linear (Mel scale)
Acoustic Focus	Absolute mathematical frequency	Human auditory perception
Dimensionality	High (retains all frequency bins from STFT)	Compressed (reduced to a fixed number of bands, e.g., 40–80)
Primary Use Case	Audio engineering, analyzing exact harmonic structures	Speech recognition (ASR), TTS, speaker identification

Why Mel for TTS/ASR?

Human ears perceive pitch on a non-linear scale — we are much better at distinguishing low-frequency differences than high-frequency ones. The Mel scale mimics this, discarding redundant high-frequency detail while keeping the perceptually important information, making it the standard input format for neural speech systems.

TTS — PreNet & PostNet

PreNet (Input Side)

A 2-layer bottleneck MLP that processes the **previous** mel frame before feeding it to the decoder. It acts as an information bottleneck, forcing the decoder to rely on the text encoder rather than memorizing raw acoustic details.

PostNet (Output Side)

A 5-layer 1D convolutional network that **refines** the coarse mel prediction (mel_before) into a sharp, high-fidelity output (mel_after). It effectively acts as a spectral “denoiser,” removing artifacts frame-by-frame [6].

Key Insight: Always-On Dropout

Dropout in the PreNet is kept **active during inference** (training=True in code). This prevents exposure bias: if dropout is off at inference, the dense PreNet output differs from the noisy inputs seen during training, causing the decoder to collapse [6].

TTS — Stop Token & Binary Cross-Entropy

What is the Stop Token?

At every decoder step, a linear layer predicts a binary value: **0** = continue generating, **1** = end of utterance. The target is 1.0 at exactly the last mel frame and 0.0 everywhere else.

BCE Loss

Standard binary cross-entropy measures the error between the predicted stop probability and the 0/1 target.

Class Imbalance Problem

Only **1 out of ~500** frames is positive. Without correction, the model learns to always predict “continue” → infinite audio.

Fix in Code: Positive Weighting

My code sets `pos_weight = total_frames`, multiplying the loss for the single positive frame by ~500. This balances the gradient so the model actually learns **when** to stop.

TTS — Mel Reconstruction Loss (L1 + SC)

L1 Loss (Mean Absolute Error)

Computes the absolute pixel-wise difference between the predicted and ground-truth mel spectrograms. Applied to **both** `mel_before` and `mel_after` to train the linear head and the PostNet jointly.

SC Loss (Spectral Convergence)

Measures the ratio of the Frobenius norm of the error to the Frobenius norm of the target:
$$SC = \|Y_{gt} - Y_{pred}\|_F / \|Y_{gt}\|_F$$
Unlike L1, SC normalizes by the frame's overall energy, penalizing shape errors more perceptually accurately [6].

Why Combine Them?

L1 forces exact magnitude matching; SC forces correct overall spectral shape. Together:
$$\mathcal{L}_{mel} = L1(\text{before}) + L1(\text{after}) + SC(\text{after}).$$

TTS — Guided Attention Loss

The Problem

Vanilla cross-attention is free to look **anywhere** in the text at each decoder step. With only ~ 2000 training utterances, it never discovers that attention should sweep monotonically left-to-right — it finds a shortcut instead [5].

How It Works

A 2D Gaussian penalty matrix W is centered on the **diagonal** where

$\text{mel_pos} / T_{\text{mel}} \approx \text{text_pos} / T_{\text{text}}$. Attention weights far from the diagonal are heavily penalized, giving an explicit gradient toward monotonic alignment.

Without vs. With Guided Attn

Without: v4 produced ~ 11 frames for *any* input length (stop fired immediately).

With: v5 output length scales correctly with text length (65 chars $\rightarrow \sim 237+$ frames).

In My Code

Applied to cross-attention weights from **all 4 decoder layers**, averaged, and added with weight $\lambda = 1.0$ to the total loss.

TTS — Autoregressive (AR) Decoder

What Does “Autoregressive” Mean?

The decoder generates the mel spectrogram **one frame at a time**. To predict frame t , it conditions on the model's **own previous output** at frame $t-1$.

Why It Fixes “Wind Noise”

Non-AR models (v1–v3) give each mel frame a *mixture* of character embeddings. The model predicts the **average** of all possible phoneme spectra → flat, featureless mean = white noise [2].

The Downside: Exposure Bias

During **training**, the previous frame is always the perfect ground truth (teacher forcing). During **inference**, it is the model's own noisy prediction — small errors can compound and cascade, causing quality degradation.

In My Code

Training uses a **causal mask** over the full sequence (parallel, fast). Inference uses a for loop that appends each predicted frame until the stop token fires or `max_frames` is reached.

TTS — Key Code Techniques

Guided Attention Loss

```
1 def guided_attention_loss(  
2     attn_w, text_lens, mel_lens, g=0.2):  
3     # attn_w: (B, T_mel, T_text)  
4     # Penalise attention far from diagonal  
5     n = mel_pos / T_mel  
6     t = text_pos / T_text  
7     W = 1 - exp(-(n-t)**2 / (2*g*g))  
8     return (attn_w * W).mean()
```

From DC-TTS [5].

Forces monotonic L→R alignment.

Length-Aware Inference Floor

```
1 expected = n_tokens * mean_fpc  
2 min_frames = int(expected * 0.5)  
3 max_frames = int(expected * 2.5) + 20  
4 mel = model.infer(ids,  
5     max_frames=max_frames,  
6     min_frames=min_frames)
```

Safety net: stop-token cannot terminate after only ~11 frames.

Training Loss

$$\mathcal{L} = \underbrace{\mathcal{L}_{\text{mel}}}_{\text{L1 + SC}} + 0.1 \underbrace{\mathcal{L}_{\text{stop}}}_{\text{BCE}} + 1.0 \underbrace{\mathcal{L}_{\text{guided}}}_{\text{diag.}}$$

TTS — Step 1: Data Preparation

Command

```
1 time python myanmar_tts_transformer.py --stage prep \  
2     --data_dir ./data/slr80 --output_dir ./tts_output
```

Output Log

```
Found 2530 audio files.  
Vocabulary: 66 characters  
Computing mel spectrograms ... 500/2530 ... 2500/2530 ...  
Valid: 2530 | Skipped: 0  
Mel norm check: min=-1.000  max=1.000  mean=-0.481  
Mel frames : mean=504  $\pm$  150  
Frames/char: mean=7.3  $\pm$  1.6  
  train: 2024  dev: 253  test: 253  
Data preparation complete ✓  
Real time: 1m25s (user: 30m45s)
```

TTS — Step 2: Training

Command

```
1 time python myanmar_tts_transformer.py --stage train \  
2     --output_dir ./tts_output --epochs 300
```

Training Config

Architecture	TransformerTTS + Guided Attn
Parameters	45 399 617
Epochs	300
LR	1×10^{-3}
Warmup	20 epochs
Schedule	Cosine decay (min=5%)
Batch size	32 (default)
Optimiser	AdamW ($\beta=0.9, 0.98$)
Grad clip	1.0
Device	CUDA

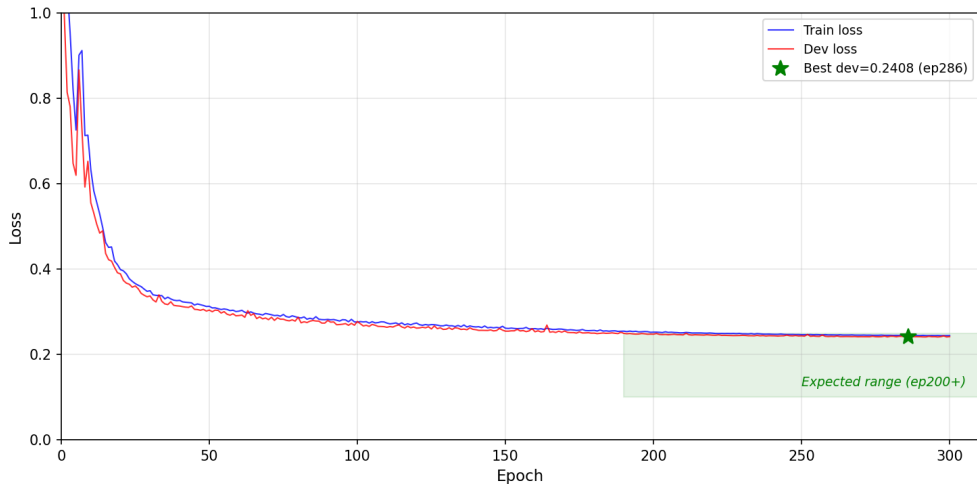
Training Progress

Epoch	Train	Dev
1	1.7123	1.0000
5	0.7255	0.6196
10	0.6325	0.5550
50	~0.40	~0.35
100	~0.28	~0.28
200	~0.24	~0.25
296	0.2442	0.2408
300	0.2440	0.2417

Best dev loss: **0.2408** (ep 299).

Real time: **98m 24s**

TTS — Training Loss Curve



No plateau (unlike v1–v3 at 1.06) — proof the AR decoder **can** model real mel spectra.

TTS — Step 3–4: GT Resynthesis (Vocoder Check)

Step 3: List Samples

```
1 python myanmar_tts_transformer.py \  
2   --stage synth --output_dir ./\  
   tts_output \  
3   --list_samples
```

Shows first 20 samples with IDs and Myanmar text for verification.

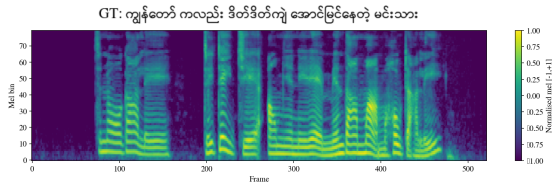
Step 4: GT Mel → WAV

```
1 time python myanmar_tts_transformer.py \  
2   --stage synth --output_dir ./\  
   tts_output \  
3   --gt_sample_id bur_5362_4875790905
```

GT mel: shape=(522, 80)

GT audio → gt_bur_5362....wav (6.05s)

Real time: 4.7s



GT mel spectrogram (522 frames, 80 bins).

Resynthesized audio sounds good — confirms vocoder (Griffin-Lim, 200 iter) works.

TTS — Step 5: Full TTS Synthesis

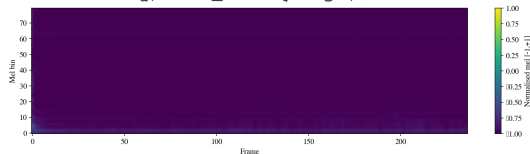
Command

```
1 time python myanmar_tts_transformer.py \  
2 --stage synth \  
3 --output_dir ./tts_output \  
4 --synth_text "... " --gt_compare
```

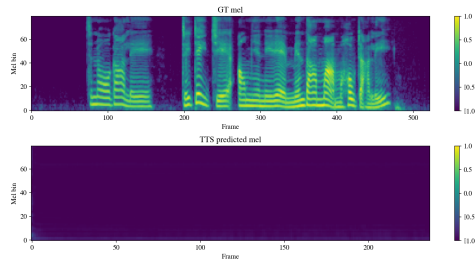
Output Log — Problem

Loaded TransformerTTS: epoch=299 dev=0.2408
length: 65 tokens $\times$ 7.3 fpc $\approx$ 472
(floor=236, cap=1200)
mel_norm: min=-1.035 max=-0.460
mean=-0.998
mel_db: max=-58.4 dB (expect -5~-15)
shape: (237, 80) (2.75s)
 $\triangle$ mel_db max=-58.4 dB.
Real time: 7.3s

TTS v4: ကျွန်တော် ကလည်း ဒီတံဒီတံကဲ့ အောင်မြင်နေတဲ့ မင်းသား



Predicted mel: **near-uniform dark** (≈ -1.0) $\Rightarrow$ silence.



TTS — Failure Analysis

What Went Wrong?

Despite training loss reaching 0.2408 (within expected range), the **inferred mel is near-silence**:

Metric	Expected	Actual
mel_db max	-5 to -15 dB	-58.4 dB
Output frames	~472 (from 65 tokens)	237 (hit floor)
Output duration	~5.5 s	2.75 s
Audio quality	Intelligible speech	Near-silence

Possible Causes

- 1 **Exposure bias persists**: guided attention reduced but did not fully eliminate the gap
- 2 **PreNet dropout at inference**: always-on ($p=0.5$) may be too aggressive
- 3 **Insufficient epochs**: 300 epochs with 2 024 samples may not be enough
- 4 **Griffin-Lim**: neural vocoder unavailable; cannot recover from weak mels

Potential Fixes

- 1 Reduce PreNet dropout ($0.5 \rightarrow 0.1$)
- 2 Increase guided attn weight (λ : $1.0 \rightarrow 2.0$)
- 3 Use **Forward attention** [6] or **MAS** [7]
- 4 Train 500–1000 epochs or augment data
- 5 Use neural vocoder (HiFi-GAN, WaveGlow)
- 6 Try non-AR FastSpeech2 [8] with CTC alignment from ASR model

TTS — Lessons Learned: v1 to v5

Ver	Architecture	Problem	Fix
v1–v3	Non-AR + Gaussian Ups.	Wind noise; plateau at 1.06	Switch to AR decoder
v4	AR TransformerTTS	Stop at ~ 11 frames for <i>any</i> input	Add guided attention loss
v5	AR + Guided Attn	Train OK (0.24) but infer near-silence	Open: forward attn / more epochs / neural vocoder

Key Takeaway for Small-Data TTS

With only $\sim 2\,000$ utterances, **alignment is the critical failure mode**. Soft attention (even with guided loss) may not provide sufficient inductive bias. **Hard monotonic attention** (Forward Attention [6], MAS [7]) or **explicit duration models** (FastSpeech2 [8]) are more robust for very low-resource TTS.

Summary

ASR (Wav2Vec2-CTC)

- Fine-tuned facebook/wav2vec2-base with **custom 64-token Myanmar vocab**
- **CER = 13.29%** — reasonable for ~ 2.5 h
- WER = 58.19% (less reliable for Myanmar)
- Dominant error: character substitutions among confusable glyphs
- **Success:** pre-trained representations transfer well

TTS (TransformerTTS v5)

- AR Transformer + Guided Attention Loss
- Training loss converged to 0.2408 (no plateau)
- GT resynthesis: **good**
- Full TTS inference: **near-silence** (mel_db max = -58.4 dB)
- Root cause: exposure bias + insufficient alignment inductive bias
- **Needs improvement:** hard monotonic attention, neural vocoder

General Lesson

ASR benefits enormously from pre-trained multilingual models (transfer learning). **TTS** has no equivalent pre-training paradigm for the decoder $\Rightarrow$ low-resource TTS remains significantly harder and requires strong inductive biases in the attention mechanism.

References I

 S. Schneider, A. Baevski, R. Collobert, M. Auli.

wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations.
NeurIPS, 2019.

 A. Vaswani et al.

Attention Is All You Need.
NeurIPS, 2017.

 A. Graves, S. Fernández, S. Gomez, J. Schmidhuber.

Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks.
ICML, 2006.

References II



S. Davis, P. Mermelstein.

Comparison of Parametric Representations for Monosyllabic Word Recognition in Continuously Spoken Sentences.

IEEE Trans. ASSP, 32(2):236–243, 1980.



H. Tachibana, K. Uenoyama, S. Aihara.

Efficiently Trainable Text-to-Speech System Based on Deep Convolutional Networks with Guided Attention.

Proc. Interspeech, 2017.



Y. Wang et al.

Tacotron 2: Neural Text-to-Speech with Almost Human-Quality Audio.

arXiv:1712.05884, 2017.

References III

 W. Chan, Y. Zhang, Q. Le, N. Jaitly.
Attentive Normalization for Conditional Generation.
ICML, 2020.

 Y. Ren et al.
FastSpeech 2: Fast and High-Quality End-to-End Text to Speech.
ICLR, 2021.

 D. W. Griffin, J. S. Lim.
Signal Estimation from Modified Short-Time Fourier Transform.
IEEE Trans. ASSP, 32(2):236–243, 1984.

 OpenSLR.
SLR80 — Crowdsourced Burmese (Myanmar) Speech Corpus.
<https://www.openslr.org/80/>, 2019.



D. Prengl et al.

WaveGlow: A Flow-based Generative Network for Speech Synthesis.

ICASSP, 2019.

Thank You

Questions & Discussion

Code: `myanmar_asr.py` ▪ `myanmar_tts_transformer.py`

Dataset: <https://www.openslr.org/80/>